

ФГБОУ ВО «Пензенский государственный университет»

Политехнический институт

Факультет вычислительной техники

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №3

по дисциплине «Логика и ОА в ИЗ»

на тему «Динамические структуры данных: очереди и стек»

Выполнили: ст. гр. 25BBB2

Перетрухин О. С.

Павлов А. Д.

Проверили:

Юрова О. В.

Деев М. В.

Пенза 2026

Цель работы: изучить принципы организации динамических структур данных на основе односвязных списков; реализовать приоритетную очередь, очередь FIFO и стек LIFO; проверить порядок добавления и удаления элементов.

1. Приоритетная очередь

Задание: Реализовать приоритетную очередь на основе односвязного списка: добавлять элемент с числовым приоритетом и выводить содержимое очереди.

Узел хранит имя объекта, приоритет и указатель на следующий узел. При добавлении выбирается позиция, при которой список остаётся упорядоченным по убыванию приоритета. Элементы с равным приоритетом сохраняют порядок поступления. Вставка и просмотр имеют сложность $O(n)$.

Листинг 1 — Программа «Приоритетная очередь»

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

struct node {
    char name[256];
    int priority;
    struct node* next;
};

struct node* head = NULL;

void add()
{
    struct node* p;
    struct node* temp;

    p = (struct node*)malloc(sizeof(struct node));

    if (p == NULL) {
        printf("Ошибка выделения памяти\n");
        return;
    }

    printf("Введите название объекта (например object_1 или object-1): ");
    scanf("%255s", p->name);

    printf("Введите приоритет (пример 1, 2, -5): ");
    scanf("%d", &p->priority);

    p->next = NULL;

    if (head == NULL || p->priority > head->priority) {
        p->next = head;
        head = p;
    }
    else {
        temp = head;

        while (temp->next != NULL &&
            temp->next->priority >= p->priority) {
            temp = temp->next;
        }
    }
}
```

```

        p->next = temp->next;
        temp->next = p;
    }
    printf("Элемент добавлен\n");
}

void review()
{
    struct node* temp = head;

    if (head == NULL) {
        printf("Очередь пуста\n");
        return;
    }

    while (temp != NULL) {
        printf("Имя: %s, приоритет: %d\n",
            temp->name, temp->priority);
        temp = temp->next;
    }
}

void clear()
{
    struct node* temp;

    while (head != NULL) {
        temp = head;
        head = head->next;
        free(temp);
    }
}

int main()
{
    setlocale(LC_ALL, "Russian");
    int choice;

    do {
        printf("\n1. Добавить элемент\n");
        printf("2. Просмотреть очередь\n");
        printf("0. Выход\n");
        printf("Ваш выбор: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                add();
                break;
            case 2:
                review();
                break;
            case 0:
                clear();
                printf("Выход\n");
                break;
            default:
                printf("Неверный выбор\n");
        }
    } while (choice != 0);

    return 0;
}

```



```
Microsoft Visual Studio — Release x64

Контрольный запуск задания 1

Добавлены элементы:
object_low, приоритет 1
object_high, приоритет 5
object_middle, приоритет 3

Просмотр очереди:
Имя: object_high,    приоритет: 5
Имя: object_middle,  приоритет: 3
Имя: object_low,     приоритет: 1
```

Рисунок 1 — Результат контрольного запуска задания 1

Очередь выведена в порядке 5, 3, 1, что подтверждает корректную вставку с учётом приоритета.

2. Очередь FIFO

Задание: Реализовать обычную очередь с операциями добавления, удаления и просмотра. Элементы должны обрабатываться по принципу «первым пришёл — первым вышел».

Для очереди используются два указателя: head на первый узел и last на последний. Новый узел присоединяется к концу, а удаление выполняется из начала. Поэтому add() и del() имеют сложность $O(1)$, а просмотр всей очереди — $O(n)$.

Листинг 2 — Программа «Очередь FIFO»

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

struct node {
    char name[256];
    struct node* next;
};

struct node* head = NULL;
struct node* last = NULL;

void add()
{
    struct node* p;

    p = (struct node*)malloc(sizeof(struct node));

    if (p == NULL) {
        printf("Ошибка выделения памяти\n");
        return;
    }

    printf("Введите название объекта:Пример (object-1 или object_1) ");
    scanf("%255s", p->name);

    p->next = NULL;

    if (head == NULL) {
        head = p;
        last = p;
    }
    else {
        last->next = p;
        last = p;
    }

    printf("Элемент добавлен\n");
}

void del()
{
    struct node* temp;

    if (head == NULL) {
        printf("Очередь пуста\n");
        return;
    }

    temp = head;
    head = head->next;

    printf("Удален элемент: %s\n", temp->name);

    free(temp);
}
```

```

        if (head == NULL) {
            last = NULL;
        }
    }
}

void review()
{
    struct node* temp = head;

    if (head == NULL) {
        printf("Очередь пуста\n");
        return;
    }

    while (temp != NULL) {
        printf("Имя: %s\n", temp->name);
        temp = temp->next;
    }
}

void clear()
{
    struct node* temp;

    while (head != NULL) {
        temp = head;
        head = head->next;
        free(temp);
    }

    last = NULL;
}

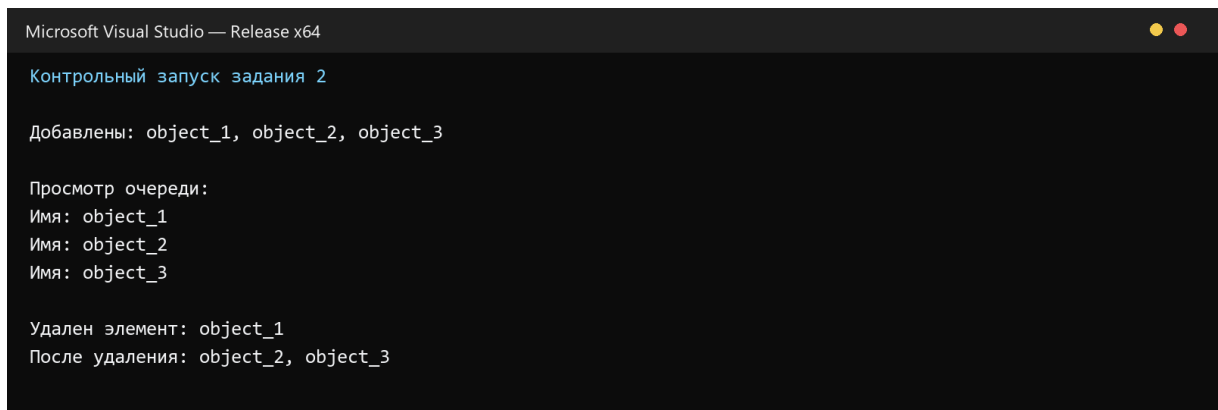
int main()
{
    setlocale(LC_ALL, "Russian");
    int choice;

    do {
        printf("\n1. Добавить элемент\n");
        printf("2. Удалить элемент\n");
        printf("3. Просмотреть очередь\n");
        printf("0. Выход\n");
        printf("Ваш выбор: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                add();
                break;
            case 2:
                del();
                break;
            case 3:
                review();
                break;
            case 0:
                clear();
                printf("Выход\n");
                break;
            default:
                printf("Неверный выбор\n");
        }
    } while (choice != 0);

    return 0;
}

```



```
Microsoft Visual Studio — Release x64

Контрольный запуск задания 2

Добавлены: object_1, object_2, object_3

Просмотр очереди:
Имя: object_1
Имя: object_2
Имя: object_3

Удален элемент: object_1
После удаления: object_2, object_3
```

Рисунок 2 — Результат контрольного запуска задания 2

Первым удалён `object_1`, который был добавлен раньше остальных. Это соответствует принципу FIFO.

3. Стек LIFO

Задание: Реализовать стек на основе односвязного списка. Программа должна добавлять элемент на вершину, удалять верхний элемент и выводить содержимое стека.

Указатель `top` хранит адрес верхнего узла. При добавлении новый узел ссылается на прежнюю вершину и сам становится вершиной. При удалении `top` перемещается к следующему узлу, а память прежней вершины освобождается. Операции `add()` и `del()` имеют сложность $O(1)$, просмотр — $O(n)$.

Листинг 3 — Программа «Стек LIFO»

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

struct node {
    char name[256];
    struct node* next;
};

struct node* top = NULL;

void add()
{
    struct node* p;

    p = (struct node*)malloc(sizeof(struct node));

    if (p == NULL) {
        printf("Ошибка выделения памяти\n");
        return;
    }

    printf("Введите название объекта (например object_1 или object-1): ");
    scanf("%255s", p->name);

    p->next = top;
    top = p;

    printf("Элемент добавлен в стек\n");
}

void del()
{
    struct node* temp;

    if (top == NULL) {
        printf("Стек пуст\n");
        return;
    }

    temp = top;
    top = top->next;

    printf("Удален элемент: %s\n", temp->name);
    free(temp);
}

void review()
{
    struct node* temp = top;

    if (top == NULL) {
        printf("Стек пуст\n");
        return;
    }
}
```

```

    printf("Содержимое стека (от вершины к основанию):\n");
    while (temp != NULL) {
        printf("Имя: %s\n", temp->name);
        temp = temp->next;
    }
}

void clear()
{
    struct node* temp;

    while (top != NULL) {
        temp = top;
        top = top->next;
        free(temp);
    }
}

int main()
{
    setlocale(LC_ALL, "Russian");
    int choice;

    do {
        printf("\n1. Добавить элемент\n");
        printf("2. Удалить верхний элемент\n");
        printf("3. Просмотреть стек\n");
        printf("0. Выход\n");
        printf("Ваш выбор: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                add();
                break;
            case 2:
                del();
                break;
            case 3:
                review();
                break;
            case 0:
                clear();
                printf("Выход\n");
                break;
            default:
                printf("Неверный выбор\n");
        }

    } while (choice != 0);

    return 0;
}

```

```

Microsoft Visual Studio — Release x64

Контрольный запуск задания 3

Добавлены: object_1, object_2, object_3

Содержимое стека (от вершины):
Имя: object_3
Имя: object_2
Имя: object_1

Удален элемент: object_3
После удаления: object_2, object_1

```

Рисунок 3 — Результат контрольного запуска задания 3

Первым удалён object_3, добавленный последним. Таким образом, стек работает по принципу LIFO.

Вывод: в ходе работы были исследованы три динамические структуры данных. Приоритетная очередь поддерживает список в порядке убывания приоритета. Обычная очередь выполняет операции по принципу FIFO, а стек — по принципу LIFO. Во всех трёх программах память для узлов выделяется динамически и освобождается после удаления элементов или при завершении работы. Контрольные запуски подтвердили правильность всех реализованных операций.